

State, Forms & Budgets

Turning a tested API into an app you'd actually use: the expense form and list, full budget management, overspend warnings, and the React state patterns that hold it all together.

SERIES

Part 2 of 3

STACK

FastAPI · SQLAlchemy ·
PostgreSQL · React

READER

Finished Part 1 — has a
working, authenticated
API

01

PART 1 · DONE

Foundations, Data & Auth

Project anatomy, the database layer, migrations, configuration, request validation, and real JWT authentication. By the end, a user could register, log in, and create expenses through a protected API.

02

YOU ARE HERE

State, Forms & Budgets

Turning the API into something a user actually touches: the expense form and list, budgets with overspend warnings, and the state-management patterns that keep a React app honest about what the server knows.

03

Analytics & the Finished Product

Aggregation queries, the charting layer, and turning a working CRUD app into something that actually answers the question “where did my money go?”

CONTENTS

01 From API to Product

The gap between endpoints that work and an app someone would actually use.

02 Lifting State Up

Why one page owns every piece of data, and every component below it doesn't.

03 Controlled Forms, Twice

One form component for both creating and editing — the pattern and the gotchas.

04 Rendering Lists Well

Keys, derived lookups, and empty states — the unglamorous work of showing data.

05 Aggregating Spend in SQL

Why "how much have I spent this month" is answered by the database, not a Python loop.

06 Budgets as Real CRUD

Full create/read/update/delete, and the two-layer defense against duplicate budgets.

07 Wiring the Overspend Warning

Tracing one expense from a form submission to a red banner, end to end.

08 Filtering: Query Params, Not Slicing

Why "show me Food in March" is a request to the server, never a client-side loop.

09 Keeping Two Resources Consistent

Why saving an expense also refreshes budgets — and what we didn't build instead.

10 Where the Simple Version Might Bite You

Honest limitations in this MVP, and what production would need to close them.

11 **Proving It Works**

Testing budgets, overspend, and filtering by hand before trusting them.

12 **Recap & What's Next**

Every concept introduced, in one table, and a look ahead to Part 3.

13 **Glossary**

Every new term this guide introduced, defined in one line.

From API to Product

Part 1 ended with an API you could fully exercise from a terminal. This part ends with one you'd hand to a friend. The gap between those two things is bigger than it looks.

By the end of Part 1, every operation this app needs already existed as a working, authenticated, tested endpoint: register, log in, create an expense, list expenses. That's not a small achievement — it's the hard, unglamorous foundation. But "the API works" and "the app works" are different claims. A user has never typed a URL into `curl`. They click a button, see a form, type a number, and expect a list on the screen to update a moment later. Everything in this part is about building the layer that turns one into the other, honestly, without hiding real state behind a friendly UI that quietly lies about what the server knows.

Two features get built end to end here, and they're deliberately entangled rather than separate: expenses and budgets. An expense form that doesn't affect a budget's overspend status isn't really finished, and a budget screen that doesn't reflect real spending is just a static form. So instead of walking through "the expense feature" and then "the budget feature" as two isolated verticals, this part is organized around the ideas that make them work *together*: where state lives, how a form talks to a list, how one write triggers two refreshes, and how a warning banner three components away learns that something changed.

CONCEPT

Nothing back-end changes its shape here. Every request this part's UI makes was already designed in Part 1 — the same `ExpenseCreate` / `ExpenseRead` schemas, the same JWT-protected routers. What's new on the backend is *scope*: budgets go from a stub to full CRUD, and expenses gain optional filters. The frontend is where the real new ground is.

Lifting State Up

Five components need to agree on what the current list of expenses is. React's answer to "who's in charge of the truth" is a specific, teachable pattern — not a library, just a placement decision.

The Expenses screen is built from five components: a form to add or edit an expense, a list to show them, a form to set a budget, a list to show budgets, and a banner that warns when any budget is exceeded. Each one is a separate file with a single job — the same separation-of-concerns instinct from Part 1's backend chapters, applied to the frontend. But separate files raise an immediate question: when the expense form successfully creates a new expense, how does the expense *list*, a completely different component, find out?

React's answer is **lifting state up**: rather than each component fetching and owning its own copy of the data, the data lives in exactly one place — the nearest component that's an ancestor of everyone who needs it — and flows *down* to children as props. When a child needs to change something, it doesn't mutate its own copy (it doesn't have one); it calls a function, also passed down as a prop, that the parent defined. This is **unidirectional data flow**: data down, events up, always in that direction, never sideways between siblings.

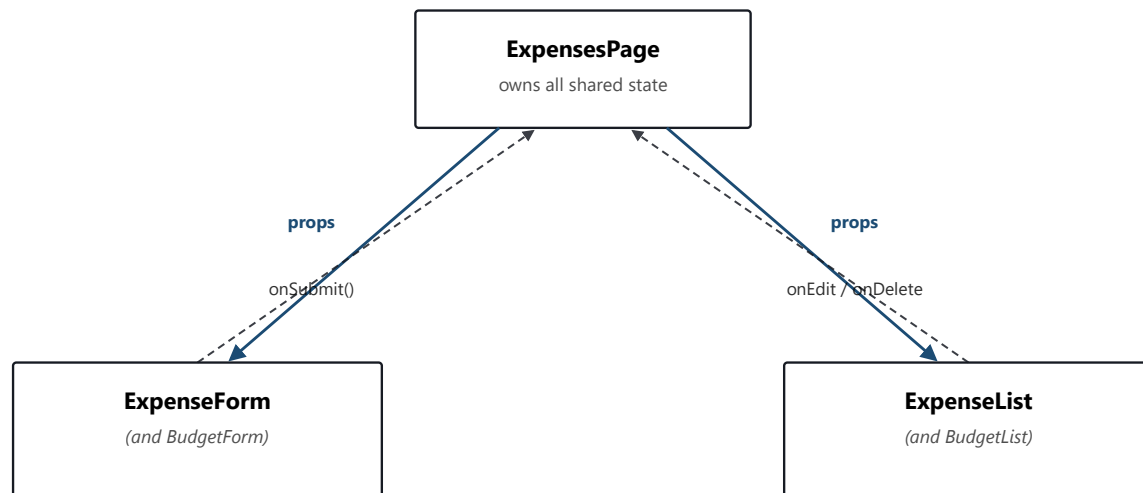


Fig. 1 — The same shape repeats for every child: data flows down as props (solid, accent), and the only way a child can change anything is to call a function the parent handed it (dashed) — the parent is always the one that actually calls `setState`. `OverspendBanner` only receives the downward arrow; it has nothing to report back, so there's no callback at all.

`ExpensesPage` is that ancestor. It holds three pieces of server data as state — `categories`, `expenses`, `budgets` — plus UI-only state that has nothing to do with the server at all: `editingExpense` (which row, if any, is currently being edited), `editingBudget`, and `filters`. Every child component is, deliberately, not very smart: `ExpenseList` doesn't know how to fetch expenses, and `ExpenseForm` doesn't know whether it's creating or editing until its parent tells it via a prop.

UNDER THE HOOD

This project reaches for plain `useState` in one page component rather than Redux, Zustand, or React Context, and that's a deliberate scope decision, not an oversight. Context (which Part 1's `AuthContext` already uses) earns its cost when state needs to be read from *many unrelated branches* of the component tree — auth status, checked on every page, is exactly that case. Expense and budget data is only ever read by children of one page, so passing it down directly as props is simpler, easier to trace, and costs nothing extra. Reach for a bigger tool only when this one stops being enough.

Controlled Forms, Twice

One form component handles both "add a new expense" and "edit this one." The trick is a single function that decides what the form should look like before the user has typed anything.

In a **controlled component**, an input's value is never left to the browser's DOM to remember — it's read from React state on every render, and every keystroke updates that state through an `onChange` handler. It's more code than an uncontrolled input, but it buys something essential here: the same `ExpenseForm` can be pre-filled with an existing expense's values just by handing it different initial state, with no special-casing in the JSX itself.

frontend/src/components/expenses/ExpenseForm.jsx (excerpt)

JSX

```
const EMPTY_FORM = { categoryId: '', amount: '', description: '', date: '' };

function toFormState(expense) {
  if (!expense) return EMPTY_FORM;
  return {
    categoryId: String(expense.category_id),
    amount: String(expense.amount),
    description: expense.description ?? '',
    date: expense.date,
  };
}

export default function ExpenseForm({ categories, initialExpense, onSubmit, onCancel }) {
  const [form, setForm] = useState(() => toFormState(initialExpense));
  const isEditing = Boolean(initialExpense);

  useEffect(() => {
    setForm(toFormState(initialExpense));
    setError('');
  }, [initialExpense]);
```

`toFormState` is the whole trick: it's a pure function that maps "no expense" to a blank form and "an expense" to that expense's values as strings. Notice every field becomes a string, even `amount` — HTML inputs only ever hold

strings, so numeric conversion happens once, deliberately, at submit time, not scattered through every keystroke handler. The `useEffect` re-runs `toFormState` whenever `initialExpense` changes, which is exactly what happens when a user clicks "Edit" on a different row while this same form instance is still mounted — without it, the form would keep showing whichever expense was being edited first.

⚠ COMMON MISTAKE

It's tempting to compute `isEditing` once and never think about it again. But `initialExpense` can change out from under an already-open form — a user clicks "Edit" on row A, then clicks "Edit" on row B before submitting. Without the `useEffect` resetting `form`, the component would still be showing row A's stale values while silently intending to `PUT` over row B's `id`, because `isEditing` and the submit handler both read the fresh `initialExpense` prop, but the visible form state wouldn't match it.

Validation happens client-side, before the network call, for a simple reason: a positive number and a non-empty category cost nothing to check locally, and catching them immediately means the user sees a specific error — "Amount must be a positive number" — instead of waiting on a round trip to learn the same thing from a `422`. This is *not* a replacement for the backend's own validation from Part 1's Pydantic schemas; it's a UX improvement layered on top of a boundary that was already safe without it. `BudgetForm` is built from the exact same shape — its own `toFormState`, its own `isEditing`, its own local validation — because it faces the identical problem: one form, two modes.

Rendering Lists Well

Turning an array of expense objects into rows on the screen looks trivial. Three small decisions in that "trivial" code are exactly the ones that go wrong first in a real app.

frontend/src/components/expenses/ExpenseList.jsx

JSX

```
export default function ExpenseList({ expenses, categories, onEdit, onDelete }) {
  const categoryNameById = new Map(categories.map((category) => [category.id, category.name]));

  if (expenses.length === 0) {
    return <p className="...">No expenses yet.</p>;
  }

  return (
    <table className="w-full">
      <tbody>
        {expenses.map((expense) => (
          <ExpenseItem
            key={expense.id}
            expense={expense}
            categoryName={categoryNameById.get(expense.category_id) ?? 'Unknown'}
            onEdit={onEdit}
            onDelete={onDelete}
          />
        ))}
      </tbody>
    </table>
  );
}
```

First: `key={expense.id}`. React uses the `key` prop to match array items across re-renders — without a stable key, deleting the third expense in a five-item list can make React think item four *became* item three, silently reusing and mismatching component state (form inputs, hover states) between rows that are no longer the same row. The database's own primary key from Part 1's `Expense` model is exactly the right value: stable, unique, and already there.

Second: `categoryNameById`. The API returns an expense's `category_id`, an integer, not its name — that's the correct shape for the backend to send (Part 1's Chapter 6 covered why request/response shapes are deliberately minimal), but a human needs to read "Food," not `3`. Rather than looking up the matching category by scanning

the `categories` array once per row (an $O(n \cdot m)$ operation as both lists grow), a `Map` is built once per render and each row does an $O(1)$ lookup. It's a small performance habit, but more importantly it's a *correctness* habit: the lookup logic exists in exactly one place instead of being copy-pasted into every row.

Third, and easy to skip entirely: the empty state. A brand new account has zero expenses, and an empty `<table>` with no rows just looks broken — there's no visual cue that anything loaded correctly at all. The explicit `if (expenses.length === 0)` check turns "nothing to show" into a deliberate, readable message instead of an accidental blank rectangle. `BudgetList` has the identical guard for the identical reason.

CONCEPT

Why does the category lookup live in the parent, not `ExpenseItem` ? If every row rebuilt its own name lookup, that's `categories.length` work repeated once per expense, every render. Building the `Map` once in `ExpenseList` and handing each `ExpenseItem` its already-resolved `categoryName` as a plain string keeps `ExpenseItem` simple — it doesn't even need to know categories exist as a concept.

Aggregating Spend in SQL

"How much have I spent on Food this month?" could be answered by loading every expense into Python and adding them up. It should never be answered that way.

Every budget response needs two numbers the **Budget** row from Part 1 doesn't store: how much has actually been spent this month in that category, and whether that's over the limit. Both are *derived* facts — they change every time a new expense is created, so storing them would mean remembering to keep them in sync everywhere an expense changes. Instead, they're computed fresh, every time, directly by the database.

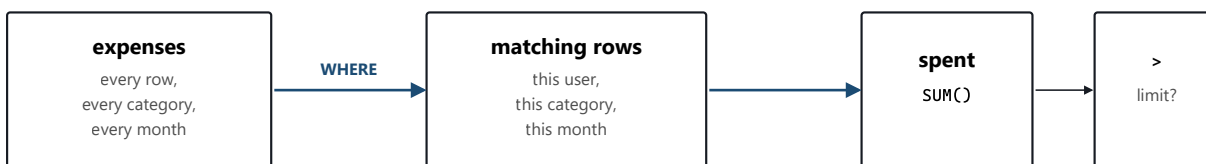


Fig. 2 — The entire calculation is one query with a filter and an aggregate, not a loop. Postgres never sends a single expense row back to Python — only the already-summed total.

```

def _spent_this_month(db: Session, user_id: int, category_id: int) → Decimal:
    today = date.today()
    total = (
        db.query(func.coalesce(func.sum(Expense.amount), 0))
        .filter(
            Expense.user_id == user_id,
            Expense.category_id == category_id,
            func.extract("year", Expense.date) == today.year,
            func.extract("month", Expense.date) == today.month,
        )
        .scalar()
    )
    return Decimal(total)

def _to_budget_read(db: Session, budget: Budget) → BudgetRead:
    spent = _spent_this_month(db, budget.user_id, budget.category_id)
    category = db.get(Category, budget.category_id)
    return BudgetRead(
        id=budget.id,
        category_id=budget.category_id,
        category_name=category.name if category else "",
        monthly_limit=budget.monthly_limit,
        spent=spent,
        is_over_budget=spent > budget.monthly_limit,
    )

```

`func.sum(Expense.amount)` is SQLAlchemy's way of writing SQL's `SUM()` aggregate rather than pulling matching rows into Python and calling the built-in `sum()` on them. For a category with three expenses the difference is invisible; for a category with three thousand, it's the difference between the database doing one pass over indexed rows internally versus the network shipping every single row across the wire just to be added together and thrown away. `func.coalesce(..., 0)` handles the edge Python's `sum()` would otherwise need an explicit check for: a brand-new budget with zero matching expenses would make `SUM()` return SQL `NULL`, not `0` — and `NULL > monthly_limit` is neither true nor false in SQL's three-valued logic. `coalesce` substitutes `0` before that comparison ever has a chance to misbehave.

`func.extract("year", ...)` and `func.extract("month", ...)` pull the calendar year and month out of a stored date, which is how "this month" is defined here: the calendar month in progress, recomputed fresh on every request rather than tied to a fixed 30-day window from when the budget was created. Every list, create, and update response runs `_to_budget_read`, so the moment you view your budgets on the first of a new month, last month's spending has already stopped counting — there's no batch job or reset step required.

💡 **CONCEPT**

Why is `spent` a computed field instead of a stored column? A stored `spent` column would need to be updated every time an expense in that category is created, edited, or deleted — three separate places that could each get it wrong, drift out of sync, or simply be forgotten by a future change. A computed value can never be stale, because it's never stored in the first place; it's recalculated from the one place that actually owns the truth — the `expenses` table.

Budgets as Real CRUD

Part 1 left `/budgets` as three unfinished stubs. Finishing them means answering a question the stub never had to: what happens when someone tries to set two budgets for the same category?

Recall the `UniqueConstraint("user_id", "category_id")` from Part 1's `Budget` model — the database itself refuses a second budget row for the same user-and-category pair. That guarantees correctness, but on its own it's a bad user experience: without an application-level check, the first sign of a duplicate budget would be Postgres raising an `IntegrityError`, which would surface to the user as an unhelpful `500 Internal Server Error`. The fix is to check first, in the application, and turn that same rule into a clear `400` with a real message — while still trusting the database constraint as the backstop.

```

def _assert_no_conflict(db: Session, current_user: User, category_id: int, exclude_budget_id:
int | None = None) → None:
    query = db.query(Budget).filter(Budget.user_id == current_user.id, Budget.category_id ==
category_id)
    if exclude_budget_id is not None:
        query = query.filter(Budget.id ≠ exclude_budget_id)
    if query.first() is not None:
        raise HTTPException(status_code=status.HTTP_400_BAD_REQUEST, detail="A budget already
exists for this category")

@router.post("/", response_model=BudgetRead, status_code=status.HTTP_201_CREATED)
def create_budget(
    payload: BudgetCreate,
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_user),
):
    if db.get(Category, payload.category_id) is None:
        raise HTTPException(status_code=status.HTTP_400_BAD_REQUEST, detail="Category not
found")
    _assert_no_conflict(db, current_user, payload.category_id)

    budget = Budget(user_id=current_user.id, category_id=payload.category_id,
monthly_limit=payload.monthly_limit)
    db.add(budget)
    db.commit()
    db.refresh(budget)
    return _to_budget_read(db, budget)

```

`exclude_budget_id` exists for exactly one caller: `update_budget`, when a user edits a budget's category. Without excluding the budget's own row from the conflict check, editing a budget while leaving its category unchanged would always find itself as a "conflict" — the row that's currently being updated already exists with that user and category. Passing `exclude_budget_id=budget.id` tells the check "ignore this specific row when looking for duplicates," which is the correct question for an edit and the wrong one for a fresh create, where there's no existing row to exclude yet.

Every response, including `create` and `update`, is built through the same `_to_budget_read` helper from Chapter 5 rather than returning the raw ORM object directly. That's the reason `BudgetRead` can safely include `spent` and `is_over_budget` even though the `Budget` table has no such columns — every route that returns a budget guarantees those fields are freshly computed at response time, never accidentally stale.

UNDER THE HOOD

Notice `_assert_no_conflict` runs a `SELECT`, and only *then* does `create_budget` run an `INSERT` — two separate round trips, not one atomic operation. Between them, in principle, another request from the same user could insert a conflicting budget first. This is the exact race condition Part 1's Chapter 3 warned about with app-level uniqueness checks. It's covered here for the same reason it was covered there: the database's `UniqueConstraint` is the real safety net — it will reject the second `INSERT` no matter what the application checked first. What this router doesn't yet do is *catch* that rare `IntegrityError` and turn it into a friendly `400` instead of a `500`. It's a known, narrow gap, and it's revisited honestly in Chapter 10.

Wiring the Overspend Warning

No single function in this app "shows the overspend banner." It emerges from six small, correct steps running in order. Tracing all six is the best way to see the whole stack working together.

A student logs a \$15 lunch in a category where they'd already hit their limit. A red banner should now be visible at the top of the page. Nothing about `OverspendBanner` itself changes when that happens — it's a small, pure function of its `budgets` prop from Chapter 4's pattern. The interesting work happens entirely upstream of it.

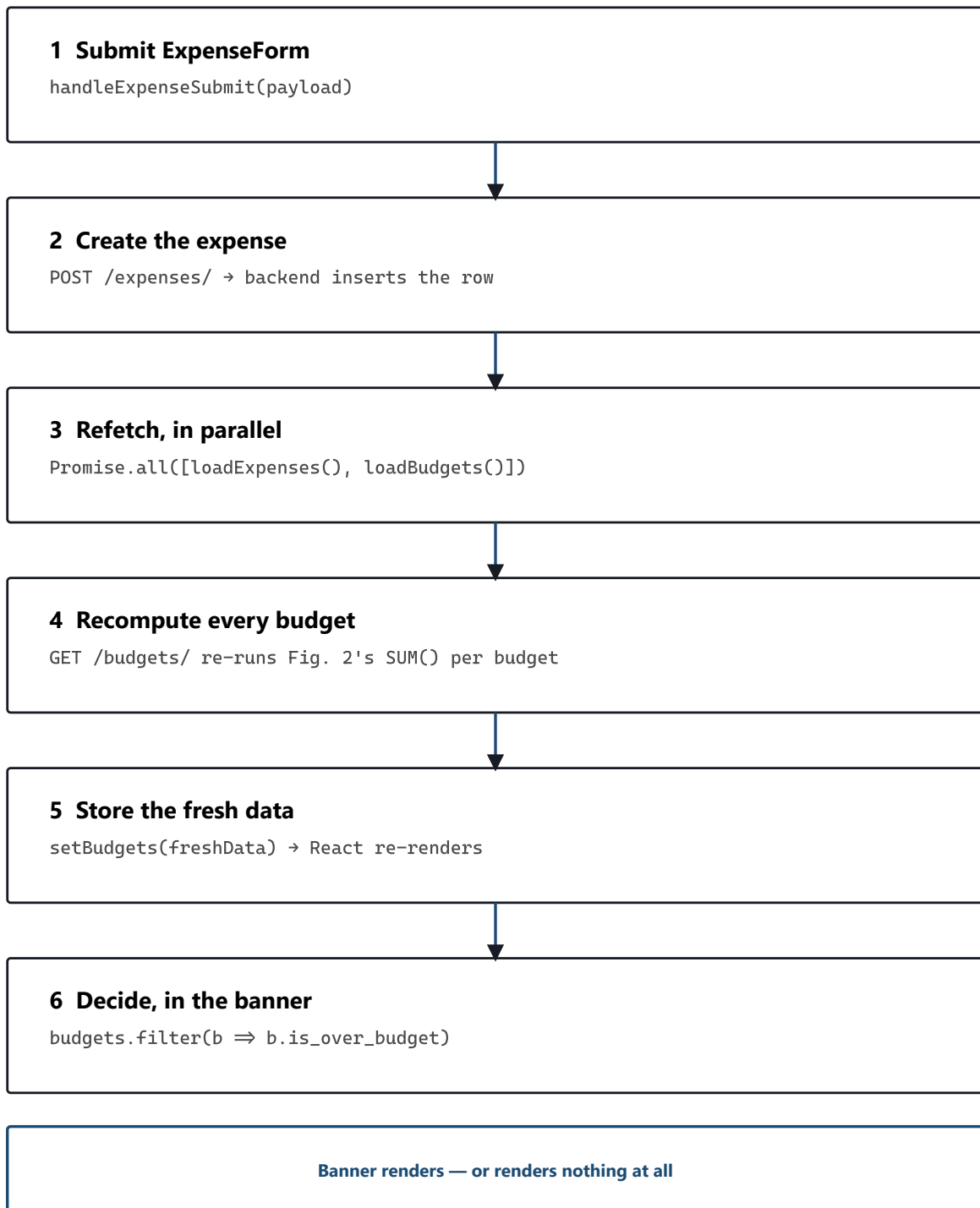


Fig. 3 — No step “knows about” overspend except the last one. Every earlier step just does its own honest job — save the expense, refetch, re-render — and the correct UI falls out the other end.

```
async function handleExpenseSubmit(payload) {
  if (editingExpense) {
    await expensesApi.update(editingExpense.id, payload);
    setEditingExpense(null);
  } else {
    await expensesApi.create(payload);
  }
  await Promise.all([loadExpenses(), loadBudgets()]);
}
```

The one line worth pausing on is `Promise.all([loadExpenses(), loadBudgets()])`. It would be simpler to only reload `expenses` after saving an expense — that's the resource that was actually written to. But `budgets`, `spent` and `is_over_budget` fields, from Chapter 5, are computed *from* the expenses table; every new expense potentially changes them, even though no budget row itself was touched. `Promise.all` fires both requests at once rather than one after another, so the user waits for the slower of the two round trips, not the sum of both — a small but genuine performance habit once a page starts making more than one request per action.

💡 CONCEPT

The banner is deliberately "dumb." `OverspendBanner` doesn't fetch anything, doesn't know an expense was ever created, and doesn't run on a timer. It receives whatever `budgets` currently is and recomputes its answer from scratch, every render. That's what makes step 6 trustworthy: there's no separate "did anything change?" bookkeeping to get wrong, because the banner never remembers anything between renders in the first place.

Filtering: Query Params, Not Slicing

"Show me Food expenses from March" could be answered by fetching everything and filtering it in the browser. That approach works right up until it quietly stops working.

Part 1's `GET /expenses/` always returned every expense the user owned. Filtering could have been bolted on entirely client-side — fetch the full list once, then use `Array.prototype.filter` in React whenever the user picks a category or date range. For a demo account with a dozen expenses, nobody would notice the difference. For a real account after a year of use, the browser would be downloading, parsing, and holding thousands of rows in memory just to throw most of them away — and every filter change would still need to re-run that in-memory filter. The fix is to make filtering a property of the *request*, not the response.

backend/app/routers/expenses.py

PYTHON

```
@router.get("/", response_model=list[ExpenseRead])
def list_expenses(
    category_id: int | None = None,
    start_date: date | None = None,
    end_date: date | None = None,
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_user),
):
    query = db.query(Expense).filter(Expense.user_id == current_user.id)

    if category_id is not None:
        query = query.filter(Expense.category_id == category_id)
    if start_date is not None:
        query = query.filter(Expense.date ≥ start_date)
    if end_date is not None:
        query = query.filter(Expense.date ≤ end_date)

    return query.order_by(Expense.date.desc(), Expense.id.desc()).all()
```

Any FastAPI path-operation parameter that isn't captured in the URL path itself and isn't the typed request body is automatically treated as a **query parameter** — the `?category_id=3&start_date=2026-08-01` part of a URL. Because all three are typed as `| None = None`, every one of them is optional; a request with none of them behaves exactly like Part 1's original unfiltered endpoint, and each filter is applied to the SQL query only when the caller actually asked for it. The database does the narrowing before a single row crosses the network, exactly as Chapter 5's aggregation did.

frontend/src/api/expenses.js

JAVASCRIPT

```
export const expensesApi = {
  getAll: async (filters = {}) => {
    const params = {};
    if (filters.categoryId) params.category_id = filters.categoryId;
    if (filters.startDate) params.start_date = filters.startDate;
    if (filters.endDate) params.end_date = filters.endDate;

    const response = await client.get('/expenses/', { params });
    return response.data;
  },
  ...
};
```

On the frontend, the filter bar is controlled state on `ExpensesPage`, the same pattern Chapter 3 used for forms — but instead of building a payload for a `POST`, every change calls `handleFilterChange`, which updates `filters` and immediately re-fetches expenses with the new values. There's no local list to slice; the list *is* whatever the server most recently returned for the current filters. "Clear filters" isn't special logic either — it just calls the same handler with an empty filter object, which the backend already treats as "return everything."

⚠ COMMON MISTAKE

Axios's `{ params }` option builds the query string automatically and handles URL-encoding correctly — a hand-built string like ``/expenses/?category_id=${categoryId}`` would work for simple values but silently mangle anything containing a character like `&` or a space if a filter value ever needed one. Letting the HTTP client own query-string construction, rather than assembling URLs with template literals, is a small habit that removes an entire category of encoding bugs before they can happen.

Keeping Two Resources Consistent

Expenses and budgets are two different API resources with two different routers, two different tables. On screen, they need to look like one consistent picture. Here's the honest version of how that's actually managed — and what a bigger app would do instead.

Chapter 7 already showed the mechanism: any change to an expense refetches both `expenses` and `budgets`. Deleting an expense follows the identical shape:

frontend/src/pages/ExpensesPage.jsx (excerpt)

JSX

```
async function handleExpenseDelete(expense) {
  if (!window.confirm(`Delete this ${expense.description || 'expense'}?`)) return;
  await expensesApi.remove(expense.id);
  if (editingExpense?.id === expense.id) setEditingExpense(null);
  await Promise.all([loadExpenses(), loadBudgets()]);
}
```

This strategy has a name: **refetch on write**. After any mutation, throw away whatever the client currently believes and ask the server for the truth again. It's the simplest correct strategy available, and "simplest correct" is worth saying plainly rather than apologizing for — there's no cache to invalidate, no risk of the UI showing something the server never actually agreed to, and no bookkeeping about which pieces of local state a given write might have affected. For an app this size, with one user acting on their own data through one open tab, it's the right choice, not a placeholder for a better one.

It's also worth naming what it costs, because every real engineering decision has a cost side: every save is followed by a visible round trip before the screen settles, and if two browser tabs are open to the same account, the second tab has no way to know the first tab just changed anything until it independently refetches. Two well-known alternatives exist, and neither was needed here:

APPROACH	WHAT IT BUYS	WHAT IT COSTS
Refetch on write (used here)	Always correct, trivial to reason about	A visible round trip after every save
Optimistic updates	The UI updates instantly, before the server confirms	Must handle rollback when the server rejects the change
WebSockets / server push	Every open tab stays in sync automatically	A persistent connection and server-side fan-out to build and maintain

CONCEPT

Don't reach for the fancier tool first. Optimistic updates and WebSockets both solve real problems — but both problems (perceived latency, multi-tab/multi-device sync) have to actually exist for a user before the extra complexity earns its cost. Refetch-on-write is not a beginner's shortcut that "real" apps grow out of; plenty of production software uses exactly this strategy forever, because the simpler tool never stopped being the right one for what the app actually needs.

Where the Simple Version Might Bite You

Part 1 had real war stories — genuine crashes with genuine tracebacks. Building this part didn't produce any drama like that, and pretending otherwise wouldn't teach anything true. What it does have is a set of known, deliberate simplifications worth naming honestly before they surprise anyone.

Every one of these is a real gap in the code as it stands right now — not hypothetical, not fixed later in this series without saying so. Reading them is itself an engineering skill: knowing exactly what an MVP does *not* guarantee is as important as knowing what it does.



KNOWN LIMITATION · THE RACE CONDITION FROM CHAPTER 6

If the same user submits two requests to create a budget for the same category at almost the same instant, both could pass `_assert_no_conflict`'s `SELECT` before either finishes its `INSERT`. Postgres's `UniqueConstraint` will still correctly reject the second `INSERT` — the data stays correct — but that rejection currently surfaces as an unhandled `IntegrityError`, which FastAPI turns into a generic `500` instead of the friendly `400` a normal conflict gets. The fix is a few lines: wrap the insert in a `try/except IntegrityError` and translate it to the same `400` response. It's not done here because it's genuinely rare for one user clicking one button — but "rare" is not "impossible," and a production system would close this before launch.



KNOWN LIMITATION · NO MULTI-TAB OR MULTI-DEVICE SYNC

As Chapter 9 named directly: if the same account is open in two tabs, one tab has no way to learn the other just deleted an expense until it performs its own action and refetches. The overspend banner in the idle tab can silently show stale information until then. This is the direct, accepted cost of choosing refetch-on-write over WebSockets.



KNOWN LIMITATION · DELETING A CATEGORY WOULD ORPHAN DATA

Part 1's `Category` model has no delete endpoint at all — categories are seeded once at startup and treated as fixed. That was a deliberate MVP simplification, not an accident, but it's worth naming the reason: expenses and budgets both hold a `category_id` foreign key with no `ON DELETE` behavior defined. If category management were added later, deleting a category with existing expenses attached to it would need an explicit decision — block the delete, reassign those expenses to "Other," or something else — that doesn't exist yet.



CONCEPT

Writing this list down is itself the practice. A codebase doesn't need to be flawless to be honest. Knowing precisely which corners were cut, and why each one was an acceptable trade for an app at this stage, is the difference between technical debt you're managing and technical debt that surprises you in production.

Proving It Works

Part 1 proved auth and expenses with four curl commands run in order. The same discipline applies here — and it's exactly what caught nothing going wrong this time, which is itself worth confirming rather than assuming.

Every claim this part makes about budgets and filtering was checked the same way Part 1 checked auth: real requests, against the real running stack, read for their actual status codes and bodies — not just skimmed for "it didn't crash."

terminal

BASH

```
# Create a budget for Food ($10/month)
curl -X POST http://localhost:8000/budgets/ \
  -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
  -d '{"category_id":1,"monthly_limit":10.00}'
# → is_over_budget: true (already spent $12.50 this month)

# Try a second budget for the same category
curl -o /dev/null -w "%{http_code}\n" -X POST http://localhost:8000/budgets/ \
  -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
  -d '{"category_id":1,"monthly_limit":50.00}'
# → 400 Bad Request

# Filter expenses to just this category
curl "http://localhost:8000/expenses/?category_id=1" \
  -H "Authorization: Bearer $TOKEN"
```

Each command is a live version of an earlier chapter's claim, checked against the real Dockerized stack rather than trusted on faith: the first proves Chapter 5's aggregation — a budget created *after* an expense already exists correctly reports `is_over_budget: true` immediately, with no separate recalculation step. The second proves Chapter 6's conflict check returns a clean `400`, not a crash. The third proves Chapter 8's filter actually narrows results at the database, not just accepts the parameter and ignores it.

Two more checks matter as much as those, precisely because they're less exciting: raising a budget's limit from \$10 to \$50 was confirmed to flip `is_over_budget` back to `false` on the very next read — proving the value really is recomputed each time, not cached from creation. And on the frontend, `npm run build` was run inside the actual

Docker image, not just a local editor's type-checker, to catch any JSX or import mistake a linter alone might miss before calling any of this done.



CONCEPT

What this testing approach does *not* prove. Curl and a production build catch wiring mistakes, wrong status codes, and broken imports — they don't confirm a human can actually click through the add-expense-then-see-a-warning flow in a real browser without something looking wrong visually. That gap is named honestly rather than glossed over: a manual pass in an actual browser is still the right next step before treating this as finished.

Recap & What's Next

Ten chapters, one page component, two fully-wired resources, and one honest list of what's still simplified. Here's everything introduced, in one table.

CONCEPT	WHERE IT LIVES IN THIS PROJECT
Lifting state up	<code>ExpensesPage</code> owns <code>categories</code> , <code>expenses</code> , <code>budgets</code>
Unidirectional data flow	Props down, callback functions up — never sideways
Controlled components	<code>ExpenseForm</code> / <code>BudgetForm</code> , one form for create <i>and</i> edit
List rendering fundamentals	Stable <code>key</code> , <code>Map</code> -based lookups, explicit empty states
SQL aggregation	<code>func.sum</code> , <code>func.coalesce</code> , <code>func.extract</code> in <code>_spent_this_month</code>
Derived vs. stored data	<code>spent</code> / <code>is_over_budget</code> computed fresh, never persisted
Race-condition awareness	<code>_assert_no_conflict</code> , backstopped by the DB's own constraint
Query-parameter filtering	Optional <code>category_id</code> / <code>start_date</code> / <code>end_date</code> on <code>GET /expenses</code>
Refetch-on-write consistency	Every expense write reloads both expenses and budgets
Naming technical debt	Chapter 10's three known, accepted limitations

What exists right now, concretely: a user can add, edit, delete, and filter expenses through a real form and list; set, edit, and delete a monthly budget per category; and see an accurate, always-current warning the moment they're over budget anywhere. What still doesn't exist is any way to see the shape of that spending over time — there's no chart, no "spending by category" breakdown, no trend line. Every number on screen right now is a list or a single total. That's exactly the gap Part 3 closes.

NEXT · PART 3

Analytics & the Finished Product

New backend endpoints aggregate spending by category and by month — more of Chapter 5's SQL-aggregation instinct, at a bigger scale. On the frontend, Recharts turns those numbers into an actual dashboard: a pie chart of spending by category first, with a bar chart and line chart if there's room. By the end of Part 3, this stops being a CRUD app and becomes the analytics tool the project was named for.

Glossary

New terms from this part only — Part 1's glossary still holds for everything introduced there.

Lifting state up

Moving shared data to the nearest common ancestor component so every child that needs it can read it as a prop.

Unidirectional data flow

Data always flows down the component tree as props; changes flow up as callback function calls.

Controlled component

A form input whose value is read from and written to React state, not left to the DOM.

Aggregate function

A SQL function like `SUM()` that reduces many rows to one value, computed inside the database.

Derived data

A value computed fresh from a source of truth rather than stored and kept in sync by hand.

Race condition

A bug that only occurs when two operations' timing overlaps in an unlucky way.

Query parameter

An optional key-value pair appended to a URL (`?category_id=3`) used to filter or modify a request.

Refetch on write

Reloading data fresh from the server after every mutation, instead of guessing how the change affects local state.

Optimistic update

Updating the UI immediately, before the server confirms, then reconciling or rolling back if it rejects the change.